

SafePulse Autonomous AI - Project Description

Overview

SafePulse is an advanced, autonomous AI-powered dashcam and SOS emergency system. It is designed to run silently in the background on a user's mobile device, continuously monitoring for crashes, speeding, and driver distraction. In the event of a critical impact, SafePulse uses an on-device machine learning model to verify the crash and initiates a robust Hybrid SOS protocol to ensure help is dispatched immediately, even without internet connectivity.

This project is divided into two main components:

- Frontend (Mobile App):** Built with Flutter, featuring background processing, sensor monitoring, and on-device AI inference.
- Backend (Server):** A Spring Boot Java application designed to receive and log emergency SOS signals.

1. Frontend: Mobile Application (Flutter)

Key Features & Architecture

- Autonomous Background Engine (SafePulseEngine):**
 - The core logic of the app runs in a background isolate using the `flutter_background_service` package. This allows continuous monitoring even when the app is closed or the screen is locked.
 - Implements a self-healing Watchdog mechanism that checks sensor health and AI pipeline integrity, automatically recovering them if they stall.
- On-Device AI Crash Detection:**
 - Uses `tflite_flutter` to run a custom TensorFlow Lite model (`crash_model.tflite`).
 - Processes high-frequency accelerometer and gyroscope data (G-Force analysis).
 - Maintains a 250-frame buffer (approx. 5 seconds of data). If an impact exceeding 3.0 Gs is detected, the AI analyzes the data window to confirm a crash, minimizing false positives.
- Hybrid SOS Protocol:**
 - Ensures absolute reliability by operating in both online and offline modes.
 - Online:** Sends an immediate HTTP POST request containing location and severity to the Spring Boot backend (`ApiService`).
 - Offline:** If the network is unavailable, it broadcasts SMS messages to a list of pre-configured emergency contacts containing a Google Maps link and timestamp (`SosService`).
 - Fallback Escalation:** If the emergency contacts are reached but do not answer the automated call, it prompts the user to escalate the situation directly to the Police (e.g., dialing 100).
- Driver Safety Monitoring:**
 - Speed Monitoring:** Tracks live speed using GPS and warns the driver if a predefined speed limit is exceeded.
 - Distraction Tracker:** Monitors screen usage while driving and calculates distraction seconds.

Tech Stack & Core Dependencies

- Framework:** Flutter (Dart)
- Machine Learning:** `tflite_flutter` (On-device Inference)
- Background Execution:** `flutter_background_service`
- Sensors & Location:** `sensors_plus`, `geolocator`
- Communications:** `telephony_fix` (SMS), `http` (API calls), `flutter_phone_direct_caller` (Direct calls)
- State Management:** `shared_preferences`, `synchronized`

2. Backend: SOS Receiver (Spring Boot)

Key Features & Architecture

- Provides a robust, lightweight RESTful API to handle incoming emergency signals.
- REST Controller (`SosController`):** Exposes an endpoint `POST /api/sos` that accepts JSON payloads containing crash details such as Event ID, Latitude, Longitude, Severity, and Location Status.
- Acts as a logging and dispatch system for online SOS triggers.

Tech Stack

- Language:** Java 17
- Framework:** Spring Boot (v3.5.14)
- Dependencies:** `spring-boot-starter-web`, `spring-boot-starter-data-jpa`, `lombok`
- Database:** Configured for MySQL (`mysql-connector-j`)

3. Key Talking Points for an Interview

As a fresher, highlighting the following aspects will demonstrate a strong understanding of modern application development:

- System Resiliency:** Discuss the **Watchdog mechanism** in `SafePulseEngine`. Explain how you understand the importance of background services not silently failing. The system actively checks for sensor death or AI stalls and automatically restarts them.
- On-Device AI (Edge Computing):** Highlight the use of TensorFlow Lite for local inference. This ensures privacy, zero latency during a crash, and functionality without an internet connection, which is critical for an emergency app.
- Hybrid Fail-Safe Architecture:** Explain the SOS fallback mechanism. Attempting online API calls first, falling back to SMS broadcasting with GPS coordinates, and ultimately executing a direct phone call to emergency services. It shows a deep understanding of fault-tolerant design.
- Hardware Interfacing:** Emphasize your knowledge of integrating with native device hardware using Flutter plugins (Accelerometer, GPS, Telephony, Direct Calling).
- Cross-Platform Background Execution:** Mention the challenges of running continuous tasks in the background on mobile devices and how

`flutter_background_service` was utilized alongside permissions handling to achieve this safely.

Summary for the Interviewer:

"SafePulse is not just a standard CRUD application; it is an intelligent, fault-tolerant edge-computing application. It leverages real-time hardware sensor data, processes it locally using an AI model to detect accidents, and employs a multi-layered, resilient SOS protocol to guarantee that help is requested regardless of internet connectivity."

4. Deep Dive: Working & Flow of the Application

The Lifecycle Flow

- App Launch & Permissions:** On startup (`HomeScreen`), the app rigorously requests a suite of critical permissions: SMS, Phone Calls, Location (Foreground & Background), and Notifications. Without these, the autonomous system cannot guarantee safety.
 - Background Initialization:** When the user taps "Start AI Sensor Monitoring", the app invokes `flutter_background_service`. The app can now be closed, and it will spawn an isolated Dart thread running `SafePulseEngine`.
 - Continuous Monitoring Loop:** The background engine orchestrates multiple services:
 - `SensorService` reads hardware sensors at 50Hz (50 times a second).
 - `LocationService` tracks speed and coordinates.
 - `Watchdog` (a timer running every 5 seconds) verifies that data is flowing correctly and restarts dead sensors if the OS suspends them.
 - Crash Detection Event:**
 - A spike in G-Force (> 3.0 Gs) triggers the `AIService`.
 - The AI evaluates 5 seconds of historical data. If the probability is > 25%, it declares a crash.
 - SOS Escalation Flow:**
 - Step 1 (Online): Sends an HTTP POST to the Spring Boot backend (`ApiService`).
 - Step 2 (Offline): Sends SMS to 6 hardcoded emergency contacts containing a Google Maps link (`SosService`).
 - Step 3 (Fallback): Initiates a direct phone call to the primary emergency contact.
 - Step 4 (Police): If the call is returned/unanswered, an on-screen dialog prompts the user to dial 100 (Police).
-

5. Fetching Hardware Data & Calculating Crashes

How Data is Fetched

The app interfaces with the mobile device's IMU (Inertial Measurement Unit) using the `sensors_plus` plugin within the `SensorService`.

- Accelerometer:** Measures acceleration across X, Y, and Z axes (including gravity).
- Gyroscope:** Measures the rate of rotation around the X, Y, and Z axes.
- Sampling Rate:** The service explicitly triggers a `Timer.periodic` every 20ms to achieve a highly consistent **50Hz sampling rate**. This raw data [`ax`, `ay`, `az`, `gx`, `gy`, `gz`] is pushed into a sliding window buffer.

Calculating Crash vs. Fall vs. Nothing

- Basic G-Force Threshold:** The app calculates the total G-Force using the Euclidean norm formula: $G = \sqrt{ax^2 + ay^2 + az^2} / 9.81$.
 - Spike Detection:** Normal driving rarely exceeds 1.5 Gs. A phone dropping might hit 3.0 Gs briefly. A car crash causes sustained, massive G-Force spikes (often > 5.0 Gs).
 - Why basic math isn't enough (The problem with Falls):** A phone dropping to the floor creates a high G-Force spike upon impact, but it lacks the prolonged deceleration and rotational chaos of a car rolling or crashing. A simple G-Force check would trigger false SOS alerts constantly.
 - The AI Solution:** Instead of relying purely on a hardcoded G-Force limit, if a spike > 3.0 Gs happens, the system passes the **entire 250-frame buffer (last 5 seconds)** to the AI model. The AI looks at the *shape* of the wave over time. A phone drop is a sharp single peak. A crash is a messy, sustained wave of multi-axis acceleration and rotation.
-

6. Artificial Intelligence (TensorFlow & CNN)

How TensorFlow is Used

The application uses the `tflite_flutter` plugin to load a pre-trained machine learning model (`assets/crash_model.tflite`).

- When an impact is detected, the 5-second matrix of data [1, 250, 6] (1 batch, 250 time steps, 6 sensor features) is reshaped and fed into the TFLite Interpreter.
- The model outputs a single float value: the probability of a crash.

Why CNN (Convolutional Neural Networks) for Time-Series?

While CNNs are famous for image processing, 1D-CNNs (One-Dimensional CNNs) are incredibly powerful for time-series sensor data.

- Pattern Recognition:** A crash has a specific "visual" signature when plotted on a graph. A 1D-CNN slides a filter across the 5 seconds of sensor data, learning to identify the local patterns of a crash (e.g., a massive deceleration followed by chaotic gyroscope tumbling).
 - Efficiency:** CNNs are computationally lightweight compared to heavy RNNs/LSTMs, making them perfect for running locally on a low-power mobile device battery without draining it.
-

7. Offline vs. Online Functionality

The core philosophy of SafePulse is that **accidents happen in dead zones** (e.g., highways, remote areas) where 4G/5G is unavailable.

- Online Mode (`ApiService`):** When internet is available, an HTTP POST request is fired to the Spring Boot backend (`http://192.168.137.1:8080/api/sos`). This allows a centralized server or dashboard to log the crash, the exact coordinates, and severity.
 - Offline Mode (`SosService`):** If the API call fails or there is no internet, the app bypasses the web entirely. It uses native telephony APIs (`telephony_fix`) to send standard SMS text messages over cellular networks. These SMS messages include a constructed Google Maps URL with the last known GPS coordinates. This ensures that as long as there is even a 2G cellular signal, the SOS will get out.
-

8. Background Services & Notification Triggers

Android strictly limits what apps can do in the background to save battery. SafePulse bypasses this legally for safety purposes using Foreground Services.

- **Foreground Service:** In `background_service.dart`, the app creates a persistent notification (ID 888: "📞 SafePulse AI Active"). Android OS sees this notification and grants the app "Foreground Service" status, preventing it from being killed to save RAM.
- **Notification Updates:** The app dynamically updates the persistent notification text based on the engine's state (e.g., "Monitoring sensors...", "⚠️ SafePulse Degraded Mode", "🚨 SOS TRIGGERED").
- **System Alerts:** The background isolate monitors system states. If it detects the user turned off GPS, it triggers Notification ID 889 warning them. If the user turns on "Battery Saver" (which throttles background CPUs), it triggers Notification ID 890 warning them that the AI might be killed by the OS.

9. Android Files Usage & Permissions

To make this deeply integrated hardware app work, native Android files had to be modified significantly.

`android/app/src/main/AndroidManifest.xml`

This file is the blueprint given to the Android OS.

- **Permissions Declared:**
 - `SEND_SMS, CALL_PHONE, READ_PHONE_STATE`: For the offline SOS protocols.
 - `ACCESS_FINE_LOCATION, ACCESS_BACKGROUND_LOCATION`: For tracking speed and generating Maps links even when the screen is locked.
 - `FOREGROUND_SERVICE, FOREGROUND_SERVICE_LOCATION`: Critical for running the SafePulseEngine isolate persistently.
 - `POST_NOTIFICATIONS`: Required in Android 13+ to show the persistent running notification.
- **Service Declarations:**
 - `Registered id.flutter.flutter_background_service.BackgroundService` with `foregroundServiceType="location"`. This explicitly tells the Android kernel that this background task is doing location tracking and shouldn't be killed.

10. Comprehensive File Breakdown

Every single file in this repository serves a specific architectural purpose. Here is the complete manifest:

Frontend (Flutter - lib/ directory)

- `main.dart`: The entry point of the UI. Initializes the Flutter binding and loads `HomeScreen`.
- `core/enums.dart`: Contains shared data structures like `EngineState`, `LogLevel`, `EngineHealthState`, and `EngineSeverity` to keep code type-safe.
- `features/safepulse/ui/home_screen.dart`: The main user interface. It requests permissions on load, displays live speed (m/s or km/h), shows the distraction tracker, contains the master START/STOP AI button, and displays a live stream of background logs. It also handles the "Call 100" dialog.
- `features/safepulse/engine/safepulse_engine.dart`: The "Brain" of the app. It instantiates all services, manages the watchdog timer, checks health, orchestrates the data pipeline from sensors to AI, and triggers the `_handleCrash()` function.
- `features/safepulse/services/sensor_service.dart`: Hooks into the native Android IMU. Standardizes polling to 50Hz and pushes raw arrays `[ax, ay, az, gx, gy, gz]`. Includes auto-restart logic if native sensors crash.
- `features/safepulse/services/location_service.dart`: Uses `geolocator` to track GPS coordinates and calculates live speed ($\text{Speed} = \text{Distance} / \text{Time}$ via GPS deltas).
- `features/safepulse/services/ai_service.dart`: Wraps the TensorFlow Lite interpreter. Maintains the 250-frame buffer. Executes `_interpreter!.run(input, output)` and returns a crash probability.
- `features/safepulse/services/sos_service.dart`: Manages the emergency response. Contains the hardcoded list of emergency contacts. Executes the Hybrid SOS (API -> SMS -> Direct Phone Call).
- `features/safepulse/services/api_service.dart`: A simple HTTP client that sends a POST request with JSON payload to the backend server.
- `features/safepulse/services/background_service.dart`: Contains the entry point `@pragma('vm:entry-point') void onStart()`. This runs in a completely separate Dart memory isolate. It updates the Android notification bar and monitors battery-saver/location settings.
- `features/safepulse/services/warning_service.dart` & `alert_service.dart`: Calculates if the user is speeding over limits or getting distracted, and plays audio/vibration alerts to warn the driver dynamically.

Configuration Files

- `pubspec.yaml`: The package manager file. Lists all dependencies like `geolocator`, `tflite_flutter`, `telephony_fix`, etc., and links the `assets/crash_model.tflite` file.
- `assets/crash_model.tflite`: The compiled, serialized binary containing the trained weights and biases of the Convolutional Neural Network.

Backend (Spring Boot - backend-springboot/ directory)

- `pom.xml`: Maven configuration file. Defines dependencies for Spring Web, Spring Data JPA, MySQL Connector, and Lombok.
- `src/main/java/com/safepulse/backend/BackendApplication.java`: The main class that bootstraps the embedded Tomcat server and Spring application context.
- `src/main/java/com/safepulse/backend/controller/SosController.java`: A REST Controller mapped to `/api/sos`. It contains a `@PostMapping` that receives the `SosRequest` object and logs the emergency (Latitude, Longitude, Severity) to the system console.
- `src/main/java/com/safepulse/backend/controller/TestController.java`: A simple health-check endpoint to verify the server is running.
- `src/main/java/com/safepulse/backend/dto/SosRequest.java`: A Data Transfer Object (DTO) using Lombok annotations (`@Data`) to automatically parse incoming JSON payloads into Java objects.